Quantitative Improvements to the Exact-Counting Contraction Algorithm

Research note prepared for Vladimir Reshetnikov
ChatGPT

26 October 2026

Abstract

We propose and prove several modifications to the tensor-contraction algorithm in OpenAI's preprint *The Exponential Time Hypothesis is False*, pinned to commit `7ef5c0cc5ca4`. An exact count of Hermite conditions reduces the taper scale from asymptotically $3M/u$ to M/u; a sharper precision allowance reduces the leading working precision from $24MD/u$ to $6MD/u$. Together with a new recursion schedule, these give the explicit contraction bound

$$(s+\mathcal{L})^{O(1)}2^{(648.4411\ldots+o(1))s/\sqrt{\log_2 s}}$$

under a normalized version of the source's interface, with polynomial space. Applying the same explicit analysis to the published schedule gives coefficient $18535.7538\ldots$. These are comparisons of upper-bound certificates, not measured speedups or lower bounds for the old algorithm. Independently, streamed finite differences improve packet coefficient extraction from $\exp(O(L\log s))$ to $\mathrm{poly}(s)2^L$ time while preserving polynomial space, including correct division at a power-of-two modulus. The accompanying implementation validates this local replacement. We do *not* prove a smaller logarithmic order for general exact #k-SAT: the bounds $2^{O_k(n\log\log n/\sqrt{\log n})}$ and, at fixed density, $2^{O_{k,D}(n/\sqrt{\log n})}$ remain unchanged. We isolate the additional mathematical advance needed for that stronger goal.

Contents

1 Outcome, scope, and the correct baseline **2**
1.1 The normalized contraction interface 3

2 Exact tapering and a smaller precision allowance **4**
2.1 Count all the Hermite conditions 4
2.2 Charge only the precision actually lost 5

3 A faster recursion schedule **5**
3.1 Packing still works 6
3.2 All retained taper scales tend to infinity 6
3.3 Inherited precision remains negligible 6

4 The explicit exponential bound **7**
4.1 Optimize the partition estimate . . . 7
4.2 Sum the costs over recursive paths . . . 7
4.3 Comparison with the published schedule . . . 8
4.4 How the schedule was chosen . . . 9

5 Polynomial-space packet coefficient extraction **9**
5.1 An exact identity over an integral lift . . . 9
5.2 Division must be done at the correct precision . . . 10
5.3 Preserve tensor identity across recursion . . . 11

6 Consequences for the SAT algorithms **11**
6.1 Exact counting . . . 11
6.2 A sharper stated approximation corollary . . . 11

7 Why a smaller logarithmic order needs another idea **12**
7.1 Further research questions . . . 12

8 Artifacts, validation, and integration **13**

A Algorithm specification **14**

B Source map **15**

1 Outcome, scope, and the correct baseline

The source version analyzed here is the October 25 preprint [1] as present in repository commit

`7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`.

Its strongest exact-counting theorem is in the quantitative appendix, incorporated into its main theorem. For fixed clause width k, it states

$$T_{\text{general}}(n,m) = (n+m)^{O_k(1)} 2^{O_k(n \log\log n/\sqrt{\log n})}, \tag{1}$$

$$T_{\text{fixed}}(n,m) = (n+m)^{O_{k,D}(1)} 2^{O_{k,D}(n/\sqrt{\log n})} \tag{2}$$

when $m \le Dn$ in the second line. Both algorithms use polynomial space. The earlier $n(\log\log n)^2/\sqrt{\log n}$ exponent still appears in the body of the source, but the appendix explicitly improves it. Removing one $\log\log n$ from that older body bound would therefore not be a new improvement to the current exact-counting theorem.

The results below have three distinct statuses.

Result	What is established
Revised contraction schedule	A smaller explicit coefficient in a proved exponential upper bound; polynomial space is retained.
Packet coefficient extraction	A strictly better asymptotic upper bound for an identified subroutine, with an executable and tested replacement.
Smaller global logarithmic order	Not established. No bound such as $2^{O(s/\log s)}$ or removal of the dense-case $\log\log n$ is claimed.

The contraction theorem in this note uses the source's packing construction, deformation identity, integrality argument, mode expansion, and recursive factorization. We prove the modifications and check their hypotheses. This is a mathematical refinement relative to that preprint, not a complete independent proof or formal verification of its claimed refutation of ETH. The small executable is a coefficient-extraction implementation, not an end-to-end SAT solver. No claim of priority is made for the classical finite-difference identity itself.

1.1 The normalized contraction interface

Let $s \geq 2$, and let $\mathcal{L}$ be the total bit length of the graph and tensor descriptions. The input is a bipartite Boolean tensor network with at most s edges, at most polynomially many vertices, and maximum degree at most $\lceil s^{9/10} \rceil$. Its entries lie in

$$\mathcal{B}_{P_0,\ell} = (\mathbb{Z}/2^{P_0}\mathbb{Z})[V]/(V^{\ell}), \qquad 1 \leq P_0, \ell \leq s.$$

An entry is computable uniformly in time $(s+\mathcal{L})^b 2^{As^{9/10}}$ and space $(s+\mathcal{L})^b$, for fixed A, b. This is the source's interface with its constant C normalized to one. A constant factor in the edge, precision, or truncation bounds can instead be absorbed by increasing s. Numerical constants below refer to this normalized s, not directly to the number of SAT variables.

Put $\lambda = \lceil \log_2 s \rceil$. We use ln for the natural logarithm. A bound with coefficient C means

$$T \leq (s+\mathcal{L})^{O(1)} 2^{(C+o(1))s/\sqrt{\lambda}}.$$

The $o(1)$ is uniform over networks satisfying the fixed interface bounds, as $s \to \infty$. Polynomial factors are kept outside the displayed exponent.

Theorem 1.1 (Quantitative contraction refinement)**.** *Under the normalized interface and the source's contraction identities, the revised algorithm described below computes the contraction deterministically in polynomial space and time*

$$(s+\mathcal{L})^{O(1)} 2^{(C_{\mathrm{new}}+o(1))s/\sqrt{\lambda}}, \qquad C_{\mathrm{new}} = \frac{2\pi\sqrt{4320}}{\ln 2\left(1-\frac{3}{128}\sqrt{12}\right)} = 648.4410666487\ldots.$$

In particular, the coefficient can be replaced by 650 *for all sufficiently large s, with fixed input-interface constants.*

The theorem improves an explicit upper-bound certificate, but preserves the order $s/\sqrt{\log s}$. Section 4.3 explains the precise comparison with the published schedule. Section 5 proves an additional, independent improvement to its packet-entry routine.

2 Exact tapering and a smaller precision allowance

We retain the source's deformation. At an active call there are M edges, packets with $L = 2^a$ slots, and the integer grid

$$S = \{-L, -L+1, \dots, L-1\}, \qquad \alpha = 2^{a+1}.$$

The deformation polynomial $F(T)$ has degree at most

$$Q = M(2L-1). \tag{3}$$

The required answer is $F(0)$ at the call's precision p. For $t \neq 0$, write $i = v_2(t)$ and $D_t = L/2^i$. The source proves that $F(t + \alpha z)$ is integral over its coefficient ring; this is essential for the precision argument.

2.1 Count all the Hermite conditions

Fix an integer $1 \leq u \leq a$. Use multiplicity

$$q_t = h \max(0, u - v_2(t))$$

at the nonzero nodes. The published proof chooses $h = \lceil 3M/u \rceil$ and uses only the L odd nodes to certify enough conditions. Counting every retained valuation level permits a smaller h.

Lemma 2.1 (Exact taper size)**.** *Define the integer*

$$W(a,u) = \sum_{i=0}^{u-1} 2^{a-i}(u-i) = L(2u - 2 + 2^{1-u})$$

and choose

$$h = \left\lceil \frac{M(2L-1)+1}{W(a,u)} \right\rceil. \tag{4}$$

Then $\sum_{t \neq 0} q_t > \deg F$*. If* $u \to \infty$*,* $u/M \to 0$*, and* $L \to \infty$*, uniformly, then*

$$h = (1 + o(1))M/u.$$

Proof. For each $0 \leq i < u \leq a$, exactly $L/2^i$ grid nodes have valuation i. Therefore the total multiplicity is $hW(a,u)$. The finite sum evaluates to $L(2u-2+2^{1-u})$, and (4) gives total multiplicity at least $Q+1$. Also,

$$\frac{Q+1}{W(a,u)} = \frac{M}{u}(1 + O(1/u) + O(1/L) + O(1/(ML))).$$

The ceiling changes the ratio to M/u by $O(u/M)$. This proves the uniform asymptotic assertion. Everything in (4) is computable by integer arithmetic; the finite-sum definition of W avoids any floating-point operation. □

This change retains the linear taper. It only removes avoidable oversampling. In particular, it does not replace Hermite interpolation by an unproved numerical approximation.

2.2 Charge only the precision actually lost

For a retained node t, let

$$E_t(T) = \prod_{q \in S \setminus \{0,t\}} \left(\frac{T-q}{t-q}\right)^{q_q}.$$

The source's exact recovery formula is

$$F(0) = \sum_{t:q_t>0} E_t(0) \left(\left[\frac{F(t+\alpha z)}{E_t(t+\alpha z)} \right]_{<q_t} \right) (-t/\alpha). \tag{5}$$

The polynomial $E_t(t+\alpha z)$ is integral with constant term one. Its truncated reciprocal is consequently integral.

Lemma 2.2 (Reduced recovery precision)**.** *The source's recovery argument remains correct with*

$$p_t = p + 6hD_t + hu(a+1). \tag{6}$$

Only the first q_t Taylor coefficients are needed.

Proof. For $i = v_2(t)$, the valuation identity used in the source is

$$v_2(E_t(0)) = D_t \sum_{j=1}^{a-i} \frac{j}{2^j} (q(i+j) - q(i)), \qquad q(i) = h \max(0, u-i).$$

Since $q(i+j) - q(i) \geq -hj$ and $\sum_{j \geq 1} j^2/2^j = 6$,

$$v_2(E_t(0)) \geq -6hD_t. \tag{7}$$

Forming the truncated quotient in (5) loses no precision. Evaluating its coefficient of z^j, where $j < q_t$, at $-t/\alpha$ loses $(a+1-i)j$ powers of two. This is at most $hu(a+1)$. Hence each coefficient error at precision (6), after the multiplier and evaluation, is divisible by 2^p. Adding errors loses no further precision. The exact sum is integral by the source's deformation identity, so reduction modulo 2^p gives the desired answer. An additional guard bit is permitted, but is unnecessary. □

All rational recovery multipliers still have polynomial bit length: the number of nodes, the multiplicities, and a remain polynomially bounded. Thus this precision refinement does not replace exact arithmetic with approximate arithmetic.

3 A faster recursion schedule

Keep the source's maximum-degree allowance $\Delta = \lceil s^{9/10} \rceil$, but use

$$r = 256, \qquad \rho = 6/r = 3/128, \qquad H = \lceil s/\lambda \rceil, \tag{8}$$

and, at depth d, use

$$u_d = \left\lfloor \frac{\lambda}{720 \cdot 12^d} \right\rfloor, \qquad a_d = 2 \sum_{j=0}^{d} u_j, \qquad a_{-1} = 0. \tag{9}$$

Calls with $M \leq H$ are evaluated directly. At an active call use $a = a_d$, $u = u_d$, $L = 2^a$, and $D_{\mathrm{arr}} = r^{2a}$. The integer parameter conditions can be checked. A finite range of exceptional sizes is handled by direct enumeration, as in the source. No claim about a useful practical crossover size is made.

3.1 Packing still works

The geometric sum in (9) gives

$$a_d \le \frac{2\lambda}{720}\frac{12}{11} = \frac{\lambda}{330}.$$

Since $2^\lambda < 2s$, the following estimates include only absolute constant factors:

$$L = O(s^{1/330}), \qquad D_{\mathrm{arr}} = O(s^{16/330}),$$
$$D_{\mathrm{arr}}^2\Delta = O(s^{329/330}), \qquad LD_{\mathrm{arr}} = O(s^{17/330}).$$

Both latter quantities are $o(s/\lambda)$. Therefore every active call eventually satisfies all three source packing conditions

$$M \ge 2D_{\mathrm{arr}}^2\Delta, \qquad M \ge LD_{\mathrm{arr}}, \qquad 2L \le \Delta. \tag{10}$$

The packing construction itself is unchanged. Its conditional-expectation argument and uniform label-class counts work for any fixed integer $r \ge 2$; they do not require $r = 64$. It gives at most $3M/L$ packets and at most $3M/r$ ports of either side in a group. A child therefore has at most

$$6M/r = \rho M$$

edges. Old vertex degrees are unchanged; each new packet vertex has at most $2L \le \Delta$ ports. This closes the degree and shrinkage invariants.

3.2 All retained taper scales tend to infinity

An active depth-d call has

$$s/\lambda < M \le s\rho^d.$$

It follows that $d \le \log_{1/\rho}\lambda + O(1)$. Consequently

$$u_d = \Omega(\lambda^\eta), \qquad \eta = 1 - \frac{\ln 12}{\ln(128/3)} = 0.3379616482\ldots > 0. \tag{11}$$

The floor is a uniform relative $o(1)$ perturbation. Also $u_d/M \to 0$ uniformly. Lemma 2.1 is therefore applicable throughout the active recursion, not merely at its root.

3.3 Inherited precision remains negligible

The source evaluates in an extension with residue field of size $2^{4\lambda}$ and with $V^\ell = 0$. Its entrywise canonical-lift convention is retained. Along a path, the increments in (6) give

$$p \le s\lambda^{O(1)}2^{a_{d-1}}. \tag{12}$$

For example, bound each preceding increment by $s\lambda^{O(1)}2^{a_j}$ and sum over $O(\log\lambda)$ levels. At a retained node $i < u_d$,

$$D_t = 2^{a_d - i} \ge 2^{a_{d-1}+u_d+1}.$$

Using $M > s/\lambda$ and (11),

$$\frac{p}{MD_t/u_d} \le \lambda^{O(1)}2^{-u_d} = o(1).$$

Similarly,

$$\frac{hu_d(a_d+1)}{MD_t/u_d} = O\Big(\frac{u_d(a_d+1)}{D_t}\Big) = o(1).$$

Together with the exact taper size this yields the uniform estimate

$$p_t = (6+o(1))MD_t/u_d. \tag{13}$$

All precisions are polynomial in s, since $a_d \le \lambda/330$. The interpolation degree $M(2L-1)$ is $O(s^{331/330}) < s^4$ eventually, so the source's existing residue-field extension still supplies enough distinct interpolation points.

4 The explicit exponential bound

4.1 Optimize the partition estimate

The source's mode expansion has at most $6M/D_t$ variables of any positive integer weight, and retains exponent tuples satisfying

$$\sum_Z w_Z\beta_Z < p_t.$$

For any $\theta > 0$, its generating-function argument gives

$$\begin{aligned}\ln N_\beta &\le \theta p_t + \frac{6M}{D_t}\sum_{j\ge1} -\ln(1-e^{-\theta j}) \\ &\le \theta p_t + \frac{6M}{D_t\theta}\sum_{h\ge1}\frac{1}{h^2} = \theta p_t + \frac{\pi^2 M}{D_t\theta}.\end{aligned}$$

Choose $\theta = \pi\sqrt{M/(D_t p_t)}$ in the analysis. Then

$$\ln N_\beta \le 2\pi\sqrt{Mp_t/D_t} = (2\pi\sqrt{6}+o(1))\frac{M}{\sqrt{u_d}}. \tag{14}$$

The algorithm never needs to calculate π or this real-valued saddle point. They are only tools for bounding the length of a finite exact enumeration.

4.2 Sum the costs over recursive paths

Uniformly over active calls at depth d,

$$\frac{M}{\sqrt{u_d}} \le (1+o(1))\frac{\sqrt{720}\,s}{\sqrt{\lambda}}(\rho\sqrt{12})^d.$$

The ratio $\rho\sqrt{12} = 3\sqrt{12}/128 < 1$. Multiplying the per-level branching bounds, or bounding the size of each level, therefore gives

$$\ln(\text{number of calls}) \le \left(\frac{2\pi\sqrt{4320}}{1-3\sqrt{12}/128}+o(1)\right)\frac{s}{\sqrt{\lambda}}. \tag{15}$$

The polynomial number of interpolation nodes, values, and groups per call contributes $O(\lambda\log\lambda)$ to this logarithm, a lower-order term. This branching count is independent of $\mathcal{L}$. Description-processing

costs remain a fixed polynomial in $s + \mathcal{L}$, rather than acquiring a power that grows with recursion depth.

A leaf examines at most 2^H Boolean assignments, where

$$H = O(s/\lambda) = o(s/\sqrt{\lambda}).$$

The original entry-evaluation allowance $2^{As^{9/10}}$ is lower order too. Even the old packet-entry routine costs only $\exp(O(s^{1/330} \log s))$ under the new schedule; Section 5 improves this further. These facts prove the time bound in Theorem 1.1 after division of (15) by $\ln 2$.

For space, use the source's depth-first traversal. Store one mode tuple, one group product, one sum, and the polynomially many interpolation data for the current call. The precisions, graph descriptions, and rational reconstruction operands have polynomial size. There are only $O(\log \lambda)$ stack levels. A retained tensor keeps its creation-time description and canonical representatives; entry routines are not recursively composed. The coefficient replacement below also uses polynomial space. This proves the space assertion.

4.3 Comparison with the published schedule

For a fair comparison, use the same optimized generating-function estimate (14) to analyze the published parameters. They are $r = 64$, $u_d = \lfloor \lambda/(131072 \cdot 2^d) \rfloor$, and $H \sim s/\sqrt{\lambda}$. The exact child-edge factor is $6/64 = 3/32$, slightly better than the source's displayed bound $1/8$. Its choices $h = \lceil 3M/u \rceil$ and $p_t = p + 8hD_t + 8M(a+1)$ give

$$p_t = (24 + o(1))MD_t/u.$$

The local logarithmic cost is consequently $(4\pi\sqrt{6} + o(1))M/\sqrt{u}$. Including the source leaf cutoff gives the certificate

$$C_{\text{source}} = \frac{4\pi\sqrt{6}}{\ln 2\sqrt{1/131072}(1 - 3\sqrt{2}/32)} + 1 = 18535.7538316684\ldots. \tag{16}$$

Quantity	Published schedule	Revised schedule
Number of groups r	64	256
Initial taper u_0/λ	$1/131072$	$1/720$
Depth decay of u_d	2^{-d}	12^{-d}
Asymptotic $h/(M/u)$	3	1
Asymptotic $p_t/(MD_t/u)$	24	6
Leaf threshold	$s/\sqrt{\lambda}$	s/λ
Base-two coefficient certificate	18535.7538	648.4411

The coefficient ratio is about 28.5851. This is *not* a 28.6-fold running-time measurement. Nor do two upper bounds establish that every implementation of the original algorithm is slower by their quotient. The source states its theorem in big-O form and does not assert (16); that number is our explicit audit of its published schedule. The defensible claim is a smaller quantitative upper-bound certificate for a concrete modified construction, together with the independent subroutine improvement below. Both global certificates have the same $s/\sqrt{\log s}$ order.

4.4 How the schedule was chosen

Within this parameter family, take fixed $r > 6$, $b > 1$, and

$$u_d = \lfloor \varepsilon \lambda / b^d \rfloor.$$

With maximum degree $s^{9/10}$, the packing inequality permits

$$\varepsilon < \frac{b-1}{80b \log_2 r}.$$

The exact taper and reduced precision give the natural-log coefficient

$$K(r, b, \varepsilon) = \frac{2\pi\sqrt{6/\varepsilon}}{1 - (6/r)\sqrt{b}},$$

provided the denominator is positive and the active taper scales tend to infinity. At the boundary of the packing inequality, optimizing over b gives $b = (r/6)^{2/3}$. This produces the limiting base-two expression

$$C_*(r) = \frac{2\pi\sqrt{480 \log_2 r}}{\ln 2 \big(1 - (6/r)^{2/3}\big)^{3/2}}.$$

Its continuous minimizer solves $(r/6)^{2/3} = 1 + 2\ln r$ and is near 251. Thus $r = 256$ and a decay factor 12 are natural simple choices. Our strict rational choice $\varepsilon = 1/720$ stays inside the packing region; its coefficient is within about 1.6% of the continuous boundary infimum for this family. This is an optimization of a particular certificate, not a global optimality claim for tensor contraction.

5 Polynomial-space packet coefficient extraction

The source defines a packet tensor by extracting a specified coefficient from a product of at most L affine row factors, then multiplying prescribed source-mode powers. Its stated entry routine enumerates polynomially many choices per factor, at cost $\exp(O(L \log s))$ and polynomial space. We replace this extraction by a streamed finite-difference formula.

5.1 An exact identity over an integral lift

Let

$$P(Z_1, \ldots, Z_t) = \prod_{r=1}^{R} \left(a_{r,0} + \sum_{j=1}^{t} a_{r,j} Z_j \right), \qquad R \le L.$$

If $|\beta| > R$, then $[Z^\beta]P = 0$. Otherwise put

$$\beta_0 = R - |\beta|, \qquad \bar{\beta} = (\beta_0, \beta_1, \ldots, \beta_t), \qquad Q(Z_0, \ldots, Z_t) = \prod_{r=1}^{R} \sum_{j=0}^{t} a_{r,j} Z_j.$$

Then Q is homogeneous of degree $R = |\bar{\beta}|$ and the requested coefficient equals $[Z^{\bar{\beta}}]Q$.

Lemma 5.1 (Homogeneous finite differences)**.** *Over any commutative ring,*

$$\left(\prod_{j=0}^{t} \beta_j! \right) [Z^{\bar{\beta}}]Q = \sum_{0 \le \gamma \le \bar{\beta}} (-1)^{R-|\gamma|} \left(\prod_{j=0}^{t} \binom{\beta_j}{\gamma_j} \right) Q(\gamma). \tag{17}$$

The sum has exactly $G = \prod_{j=0}^{t} (\beta_j + 1) \le 2^R$ evaluations.

Proof. Apply the forward difference of order β_j in coordinate j, at zero. On a monomial Z^δ, this vanishes if any $\delta_j < \beta_j$. Because Q is homogeneous of total degree $|\bar{\beta}|$, the only monomial with $\delta_j \geq \beta_j$ in every coordinate is $\delta = \bar{\beta}$. Its forward difference is $\prod_j \beta_j!$. This proves the identity without division. Finally $b + 1 \leq 2^b$ for every nonnegative integer b, so $G \leq 2^{|\bar{\beta}|} = 2^R$. □

5.2 Division must be done at the correct precision

The factorial product can be even; it is generally not invertible modulo 2^p. An implementation that divides a residue already reduced modulo 2^p is incorrect.

Use the finite free integral ring

$$A_{\mathbb{Z}} = \mathbb{Z}[U, V]/(f(U), V^\ell),$$

where f is the monic integral polynomial from the source. Its standard basis has $e\ell$ coordinates. Irreducibility of f is not required for this coefficient subroutine. Canonically lift the row-factor coefficient coordinates from precision p to integers. Write

$$B = \prod_{j=0}^{t} \beta_j! = 2^v b, \quad b \text{ odd}, \qquad v = \sum_{j=0}^{t} v_2(\beta_j!) = R - \sum_{j=0}^{t} \operatorname{popcount}(\beta_j) \leq R.$$

Compute the right side of (17) modulo 2^{p+v}. The exact identity shows that each canonical coordinate of this numerator is divisible by 2^v. Divide that integer coordinate by 2^v, multiply by $b^{-1} \bmod 2^p$, and reduce modulo 2^p. The result is the desired coefficient. Reduction of a free integral quotient ring commutes with polynomial operations, so any consistent lift gives the same final coefficient modulo 2^p.

For example, to extract $[Z^2](1+Z)^4 = 6$ modulo 8, one has $\bar{\beta} = (2, 2)$ and $B = 4$. The numerator is 24. Keeping it modulo 32 and then dividing by 4 recovers 6 mod 8. Reducing it modulo 8 first loses the information required for division.

Theorem 5.2 (Packet-entry replacement)**.** *A target coefficient of a product of $R \leq L$ affine forms in the source's coefficient ring is computable in*

$$\operatorname{poly}(s + \mathcal{L}) \prod_{j=0}^{t} (\beta_j + 1) \leq \operatorname{poly}(s + \mathcal{L}) 2^L$$

bit operations and polynomial space, using at most R extra precision bits beyond the target precision. Excessive target degree is detected directly and gives zero.

Proof. Stream the grid in (17), retaining one grid point, one product, and one numerator accumulator. Each of the R linear forms has a polynomial-size description. Ring arithmetic, binomial weights, and odd inversion take polynomial time per grid point. The factorial product has $O(R \log(R + 1))$ bits, and working ring coordinates have $p + v \leq p + R$ bits. Hence the retained state is polynomial in the parameters, which themselves are polynomial in $s + \mathcal{L}$. No table indexed by all grid points is stored. □

The bound is often much better than 2^L. A single target mode of degree b requires only $(R - b + 1)(b + 1)$ grid evaluations. The general bound 2^R is attained by the size of this grid when all positive homogenized exponents are one.

5.3 Preserve tensor identity across recursion

Each new packet tensor is tagged with its creation precision p_c. To evaluate that tensor later, run the coefficient routine with $p = p_c$, using the canonical row-factor coefficients at that precision, and return canonical coordinates modulo 2^{p_c}. Its temporary precision $p_c + v$ is internal to the computation; it does not redefine the tensor at a larger creation precision. Multiply the prescribed source-mode powers afterwards, as in the source definition. A later call lifts the resulting entry by reusing those same creation-time representatives.

This point matters because canonical lifting is not a ring homomorphism. Recomputing a tensor by silently changing all its defining coefficients to a new precision would generally describe a different lifted tensor. The delivered routine and the integration instructions keep the source's convention intact.

6 Consequences for the SAT algorithms

6.1 Exact counting

The source's fixed-density reduction builds a Boolean tensor network with $O_{k,D}(n)$ wires and bounded initial degree. Applying Theorem 1.1 gives a better explicit contraction coefficient after the normalization of s, but leaves (2) unchanged in big-O form.

For arbitrary density, the source sparsifies and constructs systems of $O(n)$ sparse parity equations. Its total-order polynomial lift has incidence degree $K = O_k(n)$ among $N = n(\log n)^{O_k(1)}$ incidences. Encoding the selected incidence subsets uses the bound

$$K \log_2\left(\frac{\mathrm{e}(N+K)}{K}\right) = O_k(n \log\log n)$$

on the wire budget. Replacing the contraction routine does not change that budget, so (1) still follows with the same logarithmic order. The constants $648.4411\ldots$ and $18535.7538\ldots$ cannot be inserted directly in front of $n \log\log n$ without also making the reduction's width-dependent constants explicit.

6.2 A sharper stated approximation corollary

There is one application in the source whose displayed logarithmic order can already be sharpened. Its randomized Max-k-SAT approximation corollary uses exponent $O_{k,\varepsilon}(n(\log\log n)^2/\sqrt{\log n})$. The accompanying proof samples $O(n/\varepsilon^2)$ clauses and creates a threshold circuit with $O_{k,\varepsilon}(n)$ gates. The circuit-to-CNF reduction has both $O_{k,\varepsilon}(n)$ variables and clauses. Thus the fixed-density theorem suffices.

Corollary 6.1. *Relative to the source's fixed-density counting theorem, its additive-εm Max-k-SAT approximation algorithm can be analyzed with time*

$$(n+m)^{O_{k,\varepsilon}(1)} 2^{O_{k,\varepsilon}(n/\sqrt{\log n})},$$

the same polynomial-space bound, and the same stated success probability $1 - 2^{-\Omega(n)}$.

Proof. Apply the fixed-density solver to each threshold formula, use binary search to determine the best sample score, and self-reduction to obtain an optimizing assignment. Polynomially many

calls do not change the exponential order. The uniform sample-approximation event and its failure probability are the same as in the source proof. □

This is an improved analysis of a stated approximation application, already implied by the source's stronger theorem. It is not a new improvement of the exact general-density bound, and it does not require the new constants proved here.

7 Why a smaller logarithmic order needs another idea

The source's central enumeration has $c = O(M/D)$ modes of each weight and budget $p = \Theta(MD/u)$. The square root in $\exp(O(M/\sqrt{u}))$ is not merely an artifact of a loose upper bound for arbitrary such mode pools.

Proposition 7.1 (An enumeration obstruction)**.** *Suppose a mode pool contains $c = \Theta(M/D)$ distinct modes of each weight $1, \dots, J$, and let $p = \Theta(MD/u)$, with $D/\sqrt{u} \to \infty$. Then, for a sufficiently small constant multiple $J = \Theta(D/\sqrt{u})$, the budget permits $\exp(\Omega(M/\sqrt{u}))$ exponent tuples even after imposing $\sum_Z \beta_Z \le M$.*

Proof. Independently choose each exponent on the cJ listed modes to be zero or one. The largest weighted sum is $cJ(J+1)/2$, which is less than p after choosing the constant in J small enough. There are $2^{cJ} = \exp(\Omega(M/\sqrt{u}))$ choices. Their ordinary degree is at most $cJ = O(M/\sqrt{u}) \le M$ eventually. □

This proposition is a lower bound for literal enumeration of a specified mode-pool model. It does not prove that such a full pool with nonzero contributions must occur on every SAT instance, and it is not a lower bound for all contraction algorithms. It does explain why just adding the obvious ordinary-degree cutoff does not prove an improved global exponent.

The packing inequality $M \ge LD_{\mathrm{arr}}$ also gives

$$u \le a \le \frac{\log_2 M}{1 + 2\log_2 r}.$$

Thus a larger storage allowance alone does not justify $u \gg \log M$ in this construction. The restriction is combinatorial.

Another tempting shortcut is to encode the mode sum as a smaller tensor network with about $E = M/\sqrt{u}$ bits and invoke the contraction theorem again. After contracting the groups internally, there are only r group tensors, and their degrees can be of order E/r. For fixed r, this is much larger than $E^{9/10}$. Choosing only polylogarithmically many groups does not fix that violation. Keeping the original small-degree internal vertices instead restores order-M wires. A correct recursive use therefore needs a new representation theorem, not just a repeated invocation of the existing result.

7.1 Further research questions

1. **Can the mode sum be evaluated without enumerating its terms?** A polynomial-space procedure costing $2^{O(M/u)}$ for the actual group tensors, with compatible recursive entry costs, would give a plausible route to $2^{O(s/\log s)}$. No such procedure is proved here. The size of the configuration set prevents deriving it from a sharper counting estimate alone.

2. **Can incidence compression preserve low algebraic degree?** The sparse polynomial lift has degree $O(n)$, but its binary subset-index encoding has $O(n \log\log n)$ wires and does not come with an $O(n)$ degree bound in those index bits. A representation preserving both a smaller wire budget and a useful degree invariant could change the dense-case exponent. The invariant would have to survive the construction of child tensors.

3. **Which packet tensors admit substantially smaller grids?** The exact coefficient grid depends on the target profile through $\prod_j(\beta_j + 1)$. Structural bounds on those profiles could improve actual entry costs far beyond the generic 2^L estimate, although entry work is already lower order in the present global theorem.

4. **Can the maximum-degree hypothesis be replaced by a structural interface?** The internally contracted group tensors are implicit and highly structured, but have large degree. An efficient contraction theorem for those particular descriptions could make the otherwise invalid mode-network bootstrap possible.

5. **Can one formalize the modified precision accounting?** Exact taper counts and the finite-difference identity are natural small formalization targets. A full formal refinement would also need the source's integral coefficient rings, canonical-lift conventions, and algorithmic cost model.

8 Artifacts, validation, and integration

The archive contains the complete LaTeX source and compiled report, exact Python reference code, reproducible checks, a pinned source manifest, and integration notes. The Python code has no third-party dependencies and is intended for Python 3.10 or later.

The coefficient routine was checked against an independent direct coefficient-multiplication implementation. The validation includes 19,542 exact comparisons across scalar residue rings, monic polynomial extensions, truncated V rings, nilpotents, empty products, excessive target degree, noncanonical representatives, and a case requiring extra precision before factorial division. The reference comparison may store a coefficient table; that storage is used only by the small-case verifier, not by the delivered streaming algorithm.

The parameter verifier independently checks exact multiplicity counts, the valuation identity for $E_t(0)$, the bound $v_2(E_t(0)) \geq -6hD_t$, and the sufficiency of the revised precision allowance on finite grids. The prototype uses the optional additional guard bit in its child-precision function; Lemma 2.2 proves that this bit is not needed. It also recomputes the displayed constants and the rational packing exponents. Its recorded output states the tested ranges. These finite checks are useful regression evidence; the all-size claims follow from the proofs above. Neither set of checks validates the full source SAT solver, establishes the source's ETH claim, or replaces formal proof checking.

The following commands reproduce the local verification and PDF:

```
python3 code/test_packet_extraction.py
python3 code/test_contraction_parameters.py
python3 code/contraction_parameters.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

To integrate the mathematical changes, replace the source's taper scale, recovery precision, root/group schedule, and leaf threshold by (4), (6), (8), and (9). Update the parameter and cost proofs using Sections 3–4. Replace only the coefficient-extraction part of each packet-entry evaluator by Theorem 5.2; retain the remaining source-mode powers and the creation-precision rule. This is a specification of a mathematical refinement, not a claim that an end-to-end executable or a Lean patch has been supplied.

A Algorithm specification

The following pseudocode uses constructive routines explicitly described in the source. It identifies which operations are changed and which are inherited.

1. Normalize the network size to s, set λ, r, H as in (8), and construct the source's coefficient-ring extension. For exceptional bounded sizes or failed parameter checks, use direct enumeration.
2. At a call with M edges and depth d, return direct contraction if $M \le H$. Otherwise compute (9) and construct the source's collision packing with $r = 256$ groups.
3. Construct the same Cauchy deformation F as the source. Compute h using the exact integer sum $W(a, u)$. For every nonzero node of positive multiplicity, use precision (6).
4. At each required residue-distinct interpolation value, form the same weighted modes. Stream the tuples with $\sum w_Z \beta_Z < p_t$. For each tuple, recursively contract the r child networks and accumulate their product modulo 2^{p_t}.
5. Evaluate each newly created packet tensor by streamed homogeneous finite differences at its own creation precision. Multiply its source-mode powers afterwards. Keep old tensor descriptions immutable.
6. Recover $F(t + \alpha z)$ by the source's unit-difference interpolation, then recover $F(0) \bmod 2^p$ from the first q_t Taylor coefficients using (5). Return canonical coordinates at the call's precision.

For polynomial-space execution, all tuple, interpolation, and child loops are depth first. No collection of all child answers or all coefficient-grid points is materialized.

B Source map

The source manifest records the full pinned URLs and checksums of the files inspected. The following source labels identify the mathematical dependencies without relying on page numbers that may change after recompilation.

Source file	Labels or passages used
`main_full_version.tex`	`thm:main`; `lem:total-order`; `lem:sparsify`; `cor:max-sat-approximation`.
`quantitative-reduction.tex`	`thm:tensor-contraction`; `eq:sharp-incidence-degree`; `eq:sharp-wire-budget`; the fixed- and arbitrary-density reductions.
`quantitative-contraction.tex`	`lem:tensor-packets`; `lem:tensor-integrality`; `lem:tapered-hermite`; `eq:tensor-hermite-loss`; `eq:tensor-mode-multiplicity`; `eq:tensor-recursion-identity`; creation-time canonical lifts.

References

[1] OpenAI, *The Exponential Time Hypothesis is False*, 17 October 2026, revised 25 October 2026. Version inspected at repository commit `7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`.
Pinned preprint directory.
Quantitative contraction source; quantitative reductions; main manuscript source.